JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface PosixFileAttributeView

All Superinterfaces:

`AttributeView`, `BasicFileAttributeView`, `FileAttributeView`, `FileOwnerAttributeView`

```
public interface PosixFileAttributeView
extends BasicFileAttributeView, FileOwnerAttributeView
```

A file attribute view that provides a view of the file attributes commonly associated with files on file systems used by operating systems that implement the Portable Operating System Interface (POSIX) family of standards.

Operating systems that implement the POSIX family of standards commonly use file systems that have a file *owner, group-owner,* and related *access permissions*. This file attribute view provides read and write access to these attributes.

The `readAttributes` method is used to read the file's attributes. The file `owner` is represented by a `UserPrincipal` that is the identity of the file owner for the purposes of access control. The `group-owner`, represented by a `GroupPrincipal`, is the identity of the group owner, where a group is an identity created for administrative purposes so as to determine the access rights for the members of the group.

The `permissions` attribute is a set of access permissions. This file attribute view provides access to the nine permission bits defined by the `PosixFilePermission` class. These nine permission bits determine the *read*, *write*, and *execute* access for the file owner, group, and others (others meaning identities other than the owner and members of the group). Some operating systems and file systems may provide additional permission bits but access to these other bits is not defined by this class in this release.

Usage Example: Suppose we need to print out the owner and access permissions of a file:

```
    Path file = ...
    PosixFileAttributes attrs = Files.getFileAttributeView(file, PosixFileAttribu
        .readAttributes();
    System.out.format("%s %s%n",
        attrs.owner().getName(),
        PosixFilePermissions.toString(attrs.permissions()));
```

Dynamic Access

Where dynamic access to file attributes is required, the attributes supported by this attribute view are as defined by `BasicFileAttributeView` and `FileOwnerAttributeView`, and in addition, the following attributes are supported:

Name	Type
"permissions"	`Set<PosixFilePermission>`
"group"	`GroupPrincipal`

The `getAttribute` method may be used to read any of these attributes, or any of the attributes defined by `BasicFileAttributeView` as if by invoking the `readAttributes()` method.

The `setAttribute` method may be used to update the file's last modified time, last access time or create time attributes as defined by `BasicFileAttributeView`. It may also be used to update the permissions, owner, or group-owner as if by invoking the `setPermissions`, `setOwner`, and `setGroup` methods respectively.

Setting Initial Permissions

Implementations supporting this attribute view may also support setting the initial permissions when creating a file or directory. The initial permissions are provided to the `createFile` or `createDirectory` methods as a `FileAttribute` with `name` `"posix:permissions"` and a `value` that is the set of permissions. The following example uses the `asFileAttribute` method to construct a `FileAttribute` when creating a file:

```
    Path path = ...
    Set<PosixFilePermission> perms =
        EnumSet.of(OWNER_READ, OWNER_WRITE, OWNER_EXECUTE, GROUP_READ);
    Files.createFile(path, PosixFilePermissions.asFileAttribute(perms));
```

When the access permissions are set at file creation time then the actual value of the permissions may differ from the value of the attribute object. The reasons for this are implementation specific. On UNIX systems, for example, a process has a *umask* that impacts the permission bits of newly created files. Where an implementation supports the setting of the access permissions, and the underlying file system supports access permissions, then it is required that the value of the actual access permissions will be equal or less than the value of the attribute provided to the `createFile` or `createDirectory` methods. In other words, the file may be more secure than requested.

Since:

1.7

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
`String`	**`name`**`()`	Returns the name of the attribute view.
`PosixFileAttributes`	**`readAttributes`**`()`	Reads the basic file attributes as a bulk operation.
`void`	**`setGroup`**`(GroupPrincipal group)`	Updates the file group-owner.
`void`	**`setPermissions`**`(Set<PosixFilePermission`	Updates the file permissions.

Methods declared in interface BasicFileAttributeView

`setTimes`

Methods declared in interface FileOwnerAttributeView

`getOwner`, `setOwner`

Method Details

name

`String name()`

Returns the name of the attribute view. Attribute views of this type have the name `"posix"`.

Specified by:

`name` in interface `AttributeView`

Specified by:

`name` in interface `BasicFileAttributeView`

Specified by:

`name` in interface `FileOwnerAttributeView`

Returns:

the name of the attribute view

readAttributes

```
PosixFileAttributes readAttributes()
                            throws IOException
```

Description copied from interface: BasicFileAttributeView

Reads the basic file attributes as a bulk operation.

It is implementation specific if all file attributes are read as an atomic operation with respect to other file system operations.

Specified by:

readAttributes in interface BasicFileAttributeView

Returns:

the file attributes

Throws:

IOException - if an I/O error occurs

setPermissions

```
void setPermissions(Set<PosixFilePermission> perms)
             throws IOException
```

Updates the file permissions.

Parameters:

perms - the new set of permissions

Throws:

ClassCastException - if the sets contains elements that are not of type PosixFilePermission

IOException - if an I/O error occurs

setGroup

```
void setGroup(GroupPrincipal group)
       throws IOException
```

Updates the file group-owner.

Parameters:

group - the new file group-owner

Throws:

`IOException` - if an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.